

Mini-projet 18 — Chargement dynamique des plugins

Module E · assemblage

Notions : **package** · **importlib** · **pkgutil** · **inspect** · **issubclass** · Durée : 2 h – 2 h 30 · Parcours Python — Ednir

Pourquoi ce mini-projet ?

C'est le cœur de ta boîte à outils : un dossier `plugins/` où **chaque fichier est un plugin**, chargé **automatiquement** au démarrage. Le but : ajouter un nouvel outil = déposer un fichier dans `plugins/`, **sans toucher au code principal**. C'est ta réponse du test de départ (« avec import »), en version pro. On combine tout : ta classe `Plugin` abstraite (MP07), les compréhensions, les modules.

1. Un package Python

Un **package** = un dossier de modules. La structure qu'on va construire :

```
18-plugins-dynamiques/
├─ core.py          # la classe Plugin (abstraite) + Cible
├─ plugins/
│   ├── __init__.py # fichier VIDE : marque le dossier comme package
│   ├── ping.py     # class PluginPing(Plugin)
│   └── scan.py     # class PluginScan(Plugin)
└─ chargeur.py     # découvre et charge tous les plugins
```

Le `__init__.py` **vide** : sa simple présence dit à Python « ce dossier est un package, tu peux importer ce qu'il y a dedans ». C'est le fichier qui rend l'import possible.

2. importlib — charger un module par son NOM

Jusqu'ici tu écrivais `import plugins.ping` en dur. Mais on ne connaît pas les noms à l'avance ! `importlib.import_module` importe un module dont le nom est dans une **variable** (une chaîne) :

```
import importlib
module = importlib.import_module("plugins.ping") # importe par une CHAÎNE
# module contient maintenant tout ce qui est défini dans ping.py
```

C'est ça la clé du « dynamique » : le nom du module est une **donnée**, pas du code figé. On peut donc importer des modules qu'on découvre à l'exécution.

3. pkgutil — lister les modules d'un package

Pour découvrir **quels** fichiers sont dans `plugins/` sans les nommer :

```
import pkgutil, plugins

for info in pkgutil.iter_modules(plugins.__path__):
    print(info.name)      # "ping", "scan", ... (les noms de fichiers, sans .py)
```

- `plugins.__path__` = le chemin du dossier du package (Python le connaît).
- `pkgutil.iter_modules(...)` = liste les modules trouvés dedans.
- `info.name` = le nom du module (`"ping"`) → à combiner : `f"plugins.{info.name}"` pour `import_module`.

4. inspect — trouver la classe Plugin dans un module notion clé

Une fois un module chargé, il faut y **repérer** la classe qui hérite de `Plugin`. `inspect.getmembers` liste tout ce qu'un module contient ; on filtre les classes qui sont des `Plugin`.

```
import inspect
from core import Plugin

for nom, obj in inspect.getmembers(module, inspect.isclass):
    if issubclass(obj, Plugin) and obj is not Plugin:
```

```
print("plugin trouvé :", nom) # PluginPing, PluginScan...
```

- `inspect.getmembers(module, inspect.isclass)` = toutes les classes définies dans le module.
- `issubclass(obj, Plugin)` = « cette classe hérite-t-elle de `Plugin` ? » (on ne garde que nos plugins).
- `obj is not Plugin` = on exclut la classe `Plugin` abstraite elle-même (elle est importée dans le module, mais ce n'est pas un vrai plugin).

🎯 Ta mission

Objectif : un chargeur qui découvre et instancie tous les plugins du dossier, automatiquement.

1. Crée la structure (fichiers séparés) :

- `core.py` : ta classe `Cible` + ta classe abstraite `Plugin` (MP06/MP07).
- `plugins/__init__.py` : **vide**.
- `plugins/ping.py` : `class PluginPing(Plugin)` — son `__init__(self)` appelle `super().__init__("ping")`, son `run` renvoie `f"ping {cible.adresse()}"`.
- `plugins/scan.py` : `class PluginScan(Plugin)`, sur le même modèle.

2. Écris `chargeur.py` avec `charger_plugins()` qui :

- parcourt les modules de `plugins/` (`pkgutil.iter_modules()`);
- importe chacun (`importlib.import_module()`);
- y trouve les classes qui héritent de `Plugin` (`inspect` + `issubclass`);
- les **instancie** et renvoie la liste des objets plugins.

3. Teste :

```
from core import Cible
c = Cible("10.0.0.5", 22)
for p in charger_plugins():
    print(p.nom, "->", p.run(c))
# -> ping -> ping 10.0.0.5:22
#      scan -> scan 10.0.0.5:22
```

♣ **LE test qui prouve que c'est dynamique** : crée un 3e fichier `plugins/banniere.py` avec un `PluginBanniere(Plugin)`, **SANS** toucher à `chargeur.py`, et relance. Le nouveau plugin doit apparaître **tout seul**. C'est ça, l'extensibilité.

✓ Critères de réussite

- `charger_plugins()` renvoie une liste d'objets plugins (pas des classes — bien instanciés avec `obj()`).
- La classe abstraite `Plugin` elle-même n'est PAS dans la liste (grâce à `obj is not Plugin`).
- Ajouter un fichier plugin le fait apparaître sans modifier `chargeur.py`.

Si tu bloques — indices progressifs

Étape	Indice
Imports du chargeur	<code>import importlib, pkgutil, inspect</code> + <code>import plugins</code> + <code>from core import Plugin</code> .
Boucler sur les modules	<code>for info in pkgutil.iter_modules(plugins.__path__):</code>
Importer chacun	<code>module = importlib.import_module(f"plugins.{info.name}")</code>
Trouver la classe	<code>for nom, obj in inspect.getmembers(module, inspect.isclass):</code> puis <code>if issubclass(obj, Plugin) and obj is not Plugin:</code>
Instancier	<code>trouves.append(obj())</code> — les <code>()</code> créent l'objet. <code>return trouves</code> à la fin.

Détail d'exécution : lance `chargeur.py` depuis le dossier **18-plugins-dynamiques/** (pas depuis `plugins/`), pour que `import plugins` et `from core import ...` fonctionnent. Sur Windows : `cd` dans le bon dossier d'abord.

Où ranger : `mini-projets/18-plugins-dynamiques/` (tout le dossier).

Quand c'est prêt : `git add .` → `commit` → `push`, puis « c'est poussé ». On enchaîne sur la **CLI** qui pilotera tout ça.

